The Report

VOL. 1, NO. 034

2026-03-30

COVER STORY

Maintainers of Last Resort

Geomys is an organization of professional open source maintainers, focused on a portfolio of critical Go projects.

Filippo Valsorda

For example, we are two thirds of the Go standard library cryptography maintainers, we provide the FIPS 140-3 validation of the upstream Go Cryptographic Module, and we fund the maintenance of x/crypto/ssh and staticcheck amongst others.

Our retainer clients engage us both to get access to our expertise, and so that the critical dependencies they rely on are professionally maintained. Beyond our portfolio, we sometimes act as maintainers of last resort when critical, security-relevant Go projects go unmaintained.

Recently, there were two occasions in which we stepped into this informal role:

we took over maintenance of the popular bluemonday HTML sanitizer when the maintainer chose to move on; and

we built alternative upgrade paths for the seemingly unmaintained gorilla/csrf library, by introducing a new carefully researched implementation into the standard library and creating a drop-in package replacement , after we discovered a security vulnerability in the original.

We can professionally serve in this role, including contracting external help, thanks to the sustainable funding of our retainer agreements. Our clients benefit from our maintenance efforts, and have a direct line to highlight projects in need.

id="bluemonday">bluemonday

bluemonday is the most popular HTML sanitizer in the Go ecosystem, used by thousands of applications and libraries to clean up user-generated markup before including it in web pages. Needless to say, it's a security-critical, load-bearing component.

In late 2023, the sole previous maintainer announced that their new professional circumstances were not compatible with volunteer OSS work, and that they were looking for responsible ways to wind it down. Geomys offered to take over maintenance instead.

Over 2024, Geomys worked with the maintainer to take over the project at its original location, avoiding the disruption of a deprecation, and guaranteeing a natural path for future security updates.

Since we work on Go and open source on a daily basis, the marginal load for Geomys is tiny, but there is outsized value to the community in knowing that security reports would be handled by dedicated professionals that can prioritize them appropriately.

Beyond handling security and critical issues, we are also discussing bringing on a domain subject expert on a contract basis to improve safety in edge cases and to future-proof the library further. Again, we can do that because we are sustainably funded through our retainer agreements.

This was welcomed as a great outcome by the original maintainer . The existence of a maintainer of last resort is not only beneficial to the consumers of the ecosystem, but also releases a lot of pressure from volunteer maintainers who would otherwise sometimes carry unsustainable loads out of a sense of duty.

id="gorillacsrf">gorilla/csrf

gorilla/csrf is an extremely popular Cross-Site Request Forgery protection middleware.

In December 2024, Patrick O'Doherty discovered that the library was unintentionally vulnerable to schemelessly same-site cross-origin request forgeries . This means

https://admin.example.com could be attacked by https://blog.example.com or, even worse, http://foo.example.com . Unless HTTP Strict-Transport-Security with includeSubDomains is used, any network attacker can control the latter and mount the attack. This was fixed publicly in January , but a new release (v1.7.3) and an advisory (CVE-2025-24358) weren't published until April.

Alerted by Patrick's finding, we looked into the library, and found a further issue that again allowed network attackers to mount CSRF attacks if the application used the TrustedOrigins option. We reported this to the project on April 18th, 2025; however, it hasn't been acknowledged and the project appears unmaintained. (We are publicly disclosing it as the customary 90-day deadline has lapsed, and all the upgrade paths listed below are available as of yesterday, with the release of Go 1.25.)

We tried reaching out to past maintainers via email and Slack to offer to take over the project, but unfortunately never heard back. Therefore, we set out to find other solutions to fill this critical CSRF-shaped hole in the ecosystem.

First, we researched the landscape of CSRF countermeasures, and consulted with subject experts, including some of the authors of relevant Web specifications. We found that modern browsers provide security metadata in request headers that makes it possible to reject cross-origin requests without any tokens or keys, leading to a drastically better developer experience, better security, and fewer false positives! The results of that investigation are public for other projects that may benefit from it.

Second, we proposed and introduced a new CrossOriginProtection middleware in the net/http standard library package. It is part of Go 1.25, released yesterday , and we recommend all gorilla/csrf users consider switching to it. We trust that a standard library solution will safely serve the ecosystem going forward.

For applications that are not ready to update to Go 1.25, we made a nearly-identical middleware available as a Go module, at filippo.io/csrf .

Finally, we made a drop-in replacement package for the whole gorilla/csrf API that uses the new countermeasures instead: filippo.io/csrf/gorilla . We tried to minimize any side-effects of the substitution, for example by returning random values in place of the now disused tokens, but we invite you to read the package docs.

Again, all of this is enabled by and part of the Geomys retainer contracts. If you work at a company with a critical dependency on the Go ecosystem, consider reaching out at hi@geomys.org. Regardless, you might also want to follow me on Bluesky at @filippo.abyssdomain.expert or on Mastodon at @filippo@abyssdomain.expert .

id="the-picture">The picture

Since we're talking about Geomys, here's a throwback to… last year? Was it just last year?? Anyway, we sponsored GopherCon US and set up a booth mostly to cover it with my collection of gophers and pins.

Teleport — For the past five years, attacks and compromises have been shifting from traditional malware and security breaches to identifying and compromising valid user accounts and credentials with social engineering, credential theft, or phishing. Teleport Identity is designed to eliminate weak access patterns through access monitoring, minimize attack surface with access requests, and purge unused permissions via mandatory access reviews.

Ava Labs — We at Ava Labs , maintainer of AvalancheGo (the most widely used client for interacting with the Avalanche Network), believe the sustainable maintenance and development of open source cryptographic protocols is critical to the broad adoption of blockchain technology. We are proud to support this necessary and impactful work through our ongoing sponsorship of Filippo and his team.

{

FEATURES

Announcing Rust 1.94.0

The Rust Release Team · Rust Blog

The Rust team is happy to announce a new version of Rust, 1.94.0. Rust is a programming language empowering everyone to build reliable and efficient software.

If you have a previous version of Rust installed via rustup , you can get 1.94.0 with:

$ rustup update stable

If you don't have it already, you can get rustup from the appropriate page on our website, and check out the detailed release notes for 1.94.0 .

If you'd like to help us out by testing future releases, you might consider updating locally to use the beta channel (rustup default beta) or the nightly channel (rustup default nightly). Please report any bugs you might come across!

id="what-s-in-1-94-0-stable"> What's in 1.94.0 stable

id="array-windows"> Array windows

Rust 1.94 adds array_windows , an iterating method for slices. It works just like windows but with a constant length, so the iterator items are &[T; N] rather than dynamically-sized &[T] . In many cases, the window length may even be inferred by how the iterator is used!

For example, part of one 2016 Advent of Code puzzle is looking for ABBA patterns: "two different characters followed by the reverse of that pair, such as xyyx or abba ." If we assume only ASCII characters, that could be written by sweeping windows of the byte slice like this:

fn has_abba (s : & str) -> bool { s . as_bytes ( ) . array_windows ( ) . any (| [a1 , b1 , b2 , a2] | (a1 != b1) && (a1 == a2) && (b1 == b2)) }

The destructuring argument pattern in that closure lets the compiler infer that we want windows of 4 here. If we had used the older .windows(4) iterator, then that argument would be a slice which we would have to index manually, hoping that runtime bounds-checking will be optimized away.

id="cargo-config-inclusion"> Cargo config inclusion

Cargo now supports the include key in configuration files (.cargo/config.toml), enabling better organization, sharing, and management of Cargo configurations across projects and environments. These include paths may also be marked optional if they might not be present in some circumstances, e.g. depending on local developer choices.

array of paths include = [" frodo.toml " , " samwise.toml " ,]

inline tables for more control include = [{ path = " required.toml " } , { path = " optional.toml " , optional = true } ,]

See the full include documentation for more details.

id="toml-1-1-support-in-cargo"> TOML 1.1 support in Cargo

Cargo now parses TOML v1.1 for manifests and configuration files. See the TOML release notes for detailed changes, including:

Inline tables across multiple lines and with trailing commas

\xHH and \e string escape characters

Optional seconds in times (sets to 0)

For example, a dependency like this:

serde = { version = " 1.0 " , features = [" derive "] }

... can now be written like this:

serde = { version = "1.0", features = ["derive"], }

Note that using these features in Cargo.toml will raise your development MSRV (minimum supported Rust version) to require this new Cargo parser, and third-party tools that read the manifest may also need to update their parsers. However, Cargo automatically rewrites manifests on publish to remain compatible with older parsers, so it is still possible to support an earlier MSRV for your crate's users.

id="stabilized-apis"> Stabilized APIs

::array_windows

::element_offset

LazyCell::get

LazyCell::get_mut

LazyCell::force_mut

LazyLock::get

LazyLock::get_mut

LazyLock::force_mut

impl TryFrom for usize

std::iter:: Peekable::next_if_map

std::iter:: Peekable::next_if_map_mut

x86 avx512fp16 intrinsics (excluding those that depend directly on the unstable f16 type)

AArch64 NEON fp16 intrinsics (excluding those that depend directly on the unstable f16 type)

```
}
{
```

Safeguarding dynamic configuration changes at scale

Cosmo W. Q · Airbnb Engineering

How Airbnb ships dynamic config changes safely and reliably.

By Cosmo Qiu , Bo Teng , Siyuan Zhou , Ankur Soni , Willis Harvey

Dynamic configuration is a core infrastructure capability in modern systems. It allows developers to change runtime behavior without restarting or redeploying services, even as the number of services and requests grows. In practice, that might mean rolling out a new address form for a region launch, tightening an authorization rule, or adjusting timeouts when a dependency is slow.

Like any powerful tool, dynamic configuration is a double-edged sword. While it enables fast iteration and rapid incident response, a bad change can cause regressions or even outages. This is a common challenge across the industry: balancing developer flexibility with system reliability.

In this post, we will outline the expectations of a modern dynamic configuration platform, then walk through the high-level architecture of Airbnb's dynamic config platform and how its core components work together to enable safe, flexible config changes.

Modern config platform essentials

As Airbnb's business grows, our expectations for the dynamic config platform have evolved over time through our own learnings as well as industry best practices. These shape our view of what a good dynamic config platform should provide, including:

A coherent experience for config management : The platform provides a streamlined, end-to-end experience for defining, reviewing, testing, and rolling out config changes. It covers the most common needs out of the box with rich built-in features, while still offering escape hatches for edge cases.

Strong reliability, availability and safety guarantees : All config changes are validated, reviewed, and rolled out progressively, with clear ownership and well-defined access control. Treating config as code is a key focus: config changes are versioned, reviewed, and auditable like service code, but remain dynamic at runtime. The platform itself must be highly available so that services can reliably fetch and apply configs. Changes should be observable, with support for fast rollbacks when needed.

Safe testing in isolated environments : Developers can validate config changes in isolated local or canary environments before they reach production.

Flexible multi-tenant support : In a multi-tenant platform, different tenants have different risk profiles. The platform should allow config owners to customize how their configs behave per tenant, including deployment triggers, guardrails, and rollout strategy (for example, AWS zone or Kubernetes pod percentage-based rollouts).

Fast and controlled incident response : During an incident, responders can ship emergency configs as needed with clear auditability. The platform also provides observability for config changes, so incident responders can tell what changed, who was affected, when the change was made, and who made the change. This enables them to effectively identify the culprit and take action.

High-level architecture

At Airbnb, Sitar is the internal name for our dynamic config platform. It provides a common way for teams to manage runtime behavior safely. At a high level, Sitar has four main parts: a developer-facing layer, a control plane, a data plane, and the clients and agents that run alongside application code.

The developer-facing layer is where config changes are created and reviewed. By default, configs are managed through a Git-based workflow, while a few exceptions are managed in the web interface (sitar-portal), which is also used for admin operations such as emergency deployments.

The control plane is responsible for orchestrating config changes. It enforces schema validation, ownership, and access control, and decides how each change should be rolled out: for example, which environments or AWS zone to target, what percentage of Kubernetes pods to start with, and how to progress the rollout over time. The control plane also specifies how to roll back the changes when needed, and supports routing in-flight configs to specific environments or slices of subscribers for fast testing.

The data plane provides scalable storage and efficient distribution of configs. It acts as the source of truth for config values and versions, and propagates updates to services reliably, consistently, and quickly.

On the product services side, an agent sidecar running alongside each service fetches the subscribed configs from the data plane and maintains a local cache. Client libraries inside the service then read from this cache and expose configs to application logic with fast, in-process access and optional fallbacks.

Putting these together, a typical change starts from a Git flow, proceeds through control-plane validation and rollout decisions, into the data plane for distribution, and finally to agents and client libraries that apply the config updates to application logic.

The change is always reviewed and approved before rollout.

Cosmo W. Q

Key design choices

```
}
{
```

My Home Automation setup

Bruno Rocha · SwiftRocks

I recently upgraded my smart home hardware, and I felt like writing a post describing my current setup to serve as inspiration for those wanting to get started or just interested in home automation in general.

Here I only describe my home automation setup; for information on my networking / homelab hardware, check out my separate "I upgraded all home networking equipment" post where I go into deeper details of that.

I use Home Assistant OS like many others. The way I like to describe HA is that it's an Alexa on steroids. With an Alexa, you buy smart devices, link them with the Alexa, and then setup automations to control those devices based on conditions like time, weather, and so on. But the problem with Alexas is that 1) the devices must support Alexas specifically, and 2) the automations themselves are very limited, only allowing you to do simple things.

Home Assistant doesn't have such limitations. HA is open-source and has a thriving community, meaning you can find plugins that enable integrations for pretty much anything you can think of, and if you don't, you can build such plugins yourself assuming you have the programming chops to do it! HA is very extensible, and thus perfect for power users who want to set up complex automations or integrate unusual devices in unusual ways.

For a long time, my setup used to be a simple Raspberry Pi 3b and a SD card. The HA community tells you that this is a bad idea (The 3b is weak and SD cards can die if you use them too much), but in my experience this is fine as long as you don't have too many devices / automations / integrations. It's a good starting point, and it did the trick for me for a couple of years until I started wanting to do more complicated integrations.

Nowadays, I've retired the 3b in favor of a Raspberry Pi 5 w/ 8GB RAM, with a 256GB official RPi NVMe SSD and enclosed on the Argon One V3 case. This gives HA enough power and cooling to do everything that I need with ease.

Today I run the server via a wired connection, but there was a long period of time where I was running it through WiFi. The community says that this is a bad idea for latency reasons, but I never had any such issues. While the wired connection is definitely much faster, the WiFi latency never really bothered me, so I can confirm it's totally fine to run HA over WiFi if you can't run a cable to it.

For voice control, I use an Alexa . The way this works is that HA has a plugin called Emulated Hue which allows you to trick an Alexa into thinking your HA server is a Philips Hue hub, allowing you to expose your devices and scripts to the Alexa in order to make use of its voice features. But you can also pay for HA Cloud and enjoy the official "proper" Alexa integration, which I don't because I want to keep everything running on the local network.

I also have a Sonoff ZBDongle-E USB stick plugged into the server in order to drive my Zigbee devices, which I'll mention in more detail further below.

In addition to all of this, I'm using a generic Huawei Android tablet under my TV to serve not only as a physical dashboard, but also as a digital photo album when idle:

The dashboarding functionality itself is achieved by using Wallpanel to expose the tablet's controls to HA, while the photo album logic is handled via HA's lovelace-wallpanel custom integration. I host the photos on the HA itself. For added security, since we're talking about a Chinese Android device, I also have a special rule on my router's firewall that prevents this tablet from communicating with devices outside the local network.

Right now I do not provide a mechanism to allow me to access my HA instance remotely as I don't have a good enough reason to do so, but I wanted to mention that HA has official support for Tailscale , making it very easy to do if you don't happen to have access to better choices like WireGuard.

Currently, I'm running a combination of WiFi and Zigbee devices, which is an alternate wireless protocol made specifically to be used by IoT devices that uses less energy and lays off a mesh network where the devices communicate with each other (as opposed to WiFi devices where everything goes through the router, thus creating a star network).

The reason I run this mix is just because I didn't know about Zigbee in the beginning. If I could go back in time, I would have the entire network consist of Zigbee devices because I think they are just better than WiFi ones overall. It uses significantly less energy (many Zigbee devices can run on those coin cell batteries), the mesh network allows you to have devices very far away from the server, and best of all: they work even when the WiFi is down.

When you buy Zigbee devices, usually the store will say that you need a hub to drive them, which they also sell. It's true that you need a hub, but it doesn't have to be that store's specific hub. When using something like HA, you can use a USB antenna stick like the one mentioned above and that will allow you to control any Zigbee device from any manufacturer via HA.

Here are the IoT devices that I have around my apartment, excluding things that are "smart" by default like TVs and such.

Sonoff Basic R2 (WiFi)

This is a DIY WiFi switch that you hook into "dumb" devices in order to be able to make them smart and turn them on and off via WiFi. Given a bit of skill with electronics (stripping / crimping wires), these switches are much cheaper and more durable compared to buying smart lamps or smart outlets, and I have many of these spread around the apartment!

```
}
{
```

⋆ Squashing

John Gruber · Daring Fireball

MacKenzie Sigalos, writing for CNBC, under the misleading headline " Tim Cook Squashes Retirement Rumors, Says He 'Can't Imagine Life Without Apple' ":

Asked about reports that he was preparing to step aside, Cook told ABC, "No, I didn't say that. I haven't said that. I love what I do deeply. Twenty-eight years ago, I walked into Apple, and I've loved every day of it since."

He added that he "can't imagine life without Apple."

The Good Morning America interview was with Michael Strahan, in a five-minute segment for the show . Strahan actually did a decent job. He asked Cook if Apple expects to be reimbursed for the $3+ billion dollars they spent on Trump's tariffs last year, now that the Supreme Court has ruled them invalid. (Cook says they're waiting to see what the courts say about getting that money back.) Strahan then asked a pretty pointed question about Cook's high-profile appearances alongside Trump — attending the inauguration (Strahan didn't mention that Cook paid Trump $1 million for the honor to attend), the 24-karat-gold Apple-logo trophy , attending the White House premiere of Melania . Cook answered by saying he's not political and only cares about policy, which makes sense only if you believe government policy decisions aren't political — which is to say it makes no sense. But Strahan asked, and Cook's answer speaks for itself.

But to the point of Sigalos's report on the interview for CNBC, Cook didn't "squash" anything related to his tenure at Apple in that interview. Watch for yourself . Cook correctly points out that he himself has never said anything (in public, at least) about being tired or wanting to "step back a little bit", as Strahan claimed he had read. But Cook does not refute that he might soon step aside as CEO, nor does he say he intends to remain CEO for the foreseeable future. It's an incredibly deft non-answer that would remain true if Cook steps down as CEO in two weeks, on April 1 (Apple's anniversary), and would remain true if he's still CEO five years from now. (The "can't imagine life without Apple" comment would fit like a glove if, say, he steps aside as CEO but becomes executive chairman of the board.)

This headline is journalistic malpractice from CNBC.

The rest of Sigalos's report is even worse:

The comments come after a turbulent stretch for Apple's C-suite. In December, the company lost AI chief John Giannandrea, its top lawyer and a key design executive in a single week — while chip guru Johny Srouji reportedly signaled he might leave, too.

The departures raised pointed questions about whether Cook's operational leadership style is the right fit for the artificial intelligence era.

Where to even start with this? Jiminy.

Giannandrea was shown the door after he blew it with Apple Intelligence. Cook took Giannandrea's responsibilities away almost a year ago , weeks after the company's embarrassing admission that next-generation Siri would be delayed by at least a full year. The December news was that Giannandrea was officially "retiring", but that was just Cook allowing him as graceful and dignified an exit as possible. He was effectively fired back in April or May.

Kate Adams, Apple's general counsel, just plain old retired in December after a successful nine-year stint in the role. Lisa Jackson announced her retirement as VP of environment, policy, and social initiatives, alongside Adams . Zero drama around either of their departures — just, for Apple, coincidentally bad timing.

The Alan Dye leaving for Meta thing, that was unexpected, and, to some degree, turbulent. But I have yet to speak to a single person within Apple, nor a single UI designer outside Apple, who thinks it's anything but good news for Apple that Dye jumped ship for Meta. Not just that Dye is a fraud of a UI designer. Not just that he and his inner circle have vandalized MacOS , the crown jewel of human-computer interaction. Not just that he and his team are given — or have taken — credit for innovative, high-quality work on VisionOS that really belongs to the interaction team Mike Rockwell put together for VisionOS. Not just that Dye left Apple for a rival company, period — something unheard of amongst Apple's bleed-in-six-colors executive ranks. But that he left for Meta , of all fucking companies? That's the proof that Dye (and his urban cowboy magazine-designer cohort) never belonged at Apple in the first place.

And then there's the Srouji thing, which was reported only once, by Mark Gurman at Bloomberg, and then effectively retracted two days later after Srouji shot it down with a meant-to-leak memo to his staff . My own reporting, talking to several sources close to and in some cases within Apple's executive ranks, is that there is no truth to Gurman's Bloomberg report that Srouji threatened Tim Cook that he was considering leaving Apple for a competitor.

To believe that report, you need to believe not only that Srouji is unhappy while seeing his life's work flourish, leading what is inarguably one of the most successful silicon design divisions in the history of computing, and but also that at age 62, he would consider leaving Apple not to retire but to head up chip design at another company — any of which possible destinations being a company that is years behind Apple in chip design. And you have to believe that it's a successful tactic for senior executives at Apple to get what they want from Tim Cook by threatening him with poaching offers from competing companies. And that Johny Srouji would either personally leak this to Mark Gurman, or loose-lippedly blab about it to someone who would leak it to Mark Gurman. And that Gurman reporting the already-very-difficult-to-believe story at Bloomberg, making private negotiations public and embarrassing both Cook personally and Apple as a company, would lead Tim Cook to cave in and

```
}
{
```

Story Time

Steve Yegge

So I've got all these fancy blog posts planned. More than planned, actually — they're well underway. But it's also been a busy couple of months, so nothing's really ready yet.

To make my schedule even worse, I kind of sort of got myself a little bit addicted to the writings of this one blogger. Normally I can't frigging stand blogs, including my own. Everyone always asks me what blogs I read, and I reply: "Um... do books count?" Which of course is greeted with blank stares, since nobody seems to read books anymore these days. Such a shame.

Anyway, recently I tried another fling with reddit. I'm always getting addicted to reddit for a little while, and then coming off it cold-turkey for a month or two while I reassess whether I want to be spending my life reading pun threads and upmodded lolcat pictures and conspiracy theories.

Not to mention the comments, which can individually sometimes be quite cool, but in aggregate only make sense when you read them aloud with the voice of the Comic Book Guy from the Simpsons: "I upvoted you for the appropriate uses of 'its' and 'their' in the link title, but downvoted you because your link actually appeared on a little-known German social networking site several hours ago. I feel it is important that you understand that this is not a zero-vote of abstention, but rather a single upvote and a single downvote cancelling one another out." If you read the comments with CBG's voice, a lot of them make a whole lot more sense.

No matter how many times I quit, in the end I always wind up sneaking a peek at the reddit home page in a moment of weakness, and before long I'm hooked again, following the adventures of memes about narwhals and how is babby formed and all this other stuff I have zero context for, but occasionally it's hysterically funny. I'm guessing this is more or less what heroin is like. Some days I don't even shower.

In any case, my latest reddit flirtation ultimately led to a secondary and more severe addiction to the writings of some dude who goes by the name "davesecretary". Yeah, him. He's a fucking genius. I started reading his old re-posted stories, got hooked and basically blacked out and lost about three days where nobody knew where the hell I was. Eventually I made my way to his stories about his trip to China. At that point I became actively envious of someone for I think the second or maybe third time in my adult life. I'm just not the jealous type normally, but damn that guy can write . He has a real gift.

So now I'm flat-out jealous. I'm not going to make a secret of it, either, and pretend he's started some trend and I'm just a bigger light jumping on the bandwagon or some lame shit like that. No, he writes way better than I do, and I'm just going to copy him blatantly, like Terry Brooks rewriting Lord of the Rings line-by-line badly as Shitstorm of Shannara.

I had actually already been thinking of publishing some true stories. In fact before I stumbled on davesecretary's tales, I had written down the story below about my brother Dave taking out the garbage, and I had been planning more. But Dave Secretary's stories were like the Great Pyramids next to the mud pueblo of my own ambition. He's like a force of nature; stories just surround the guy like gnats, and he spews out the funniest, most interesting possible versions of them imaginable.

So while I work on my more technically ambitious blogs-in-progress, I thought I'd kick off a series of light reading. It's all true stories. None of it is anywhere near as cool as trying to find a box bigger than Kyle's goddamn cereal box, and nothing in them is anywhere near as funny as the line "Iodine, Dave." But at least these are my own stories, and they're kinda fun to write down once you get going on them. I probably have about a hundred stories like the ones here. I'll start with nine random stories that managed to get themselves written first, in no particular order.

I hope you like at least one or two of them. If not, well, I'll read your complaints out loud as CBG, so we'll be even.

So this one time I'm when in my early 20s, a fragile time during which I'm as fat as a walrus, I'm spending way too much time thinking about the name of the Star Wars character "Dash Rendar". He's some random Star Wars dude, except he's not from one of the actual movies. He's only in the aftermarket books or games or toys or whatever.

I have no idea where I'd heard the name "Dash Rendar", but I can tell you I absolutely loathed it. Lucas names are always a little hard to believe, but this one was just too much. I guess when you're that fat it's easy to get irritated by little things. At that point in my life I was genuinely annoyed that the famous one in the family hadn't been little Pete Rendar or Floyd Rendar or whoever else with a normal name. No. It just had to be frigging Dash.

I'm telling you all this so you can understand my frame of mind for what happened next.

I'm coming up the elevator from the underground garage because I'm too obese to take the stairs, and I'm in a surly mood on account of this stupidly-named character Dash Rendar. I'm trying to understand basic human nature here: how could anyone, even a nine year old, suspend disbelief for such an idiotic name?

I'm all alone in the elevator, so on the spur of the moment I decide to pretend to be a nine year old and see if it makes any difference. So I make this muscle pose and say in my most ominous nine-year-old voice: "I am DASH RENDAR!"

Right then the elevator doors open and fucking Dash Rendar walks in. I am not making this up. If you lined up ten thousand guys and asked a hundred people to point to the one whose name in real life was most probably Dash Rendar, they'd all point to this dude.

```
}
{
```

XEmacs is Dead. Long Live XEmacs!

Steve Yegge

“We’re going to get lynched, aren’t we?” — Phouchg

And you thought I’d given up on controversial blogs. Hah!

This must be said: Jamie Zawinski is a hero. A living legend. A major powerhouse programmer who, among his many other accomplishments, wrote the original Netscape Navigator and the original XEmacs . A guy who can use the term “ downward funargs “ and then glare at you just daring you to ask him to explain it, you cretin. A dude with arguably the best cat-picture blog ever created.

I’ve never met him, but I’ve been in awe of his work since 1993-ish, a time when I was still wearing programming diapers and needing them changed about every 3 hours.

Let’s see... that would be 15 years ago. I’ve been slaving away to become a better programmer for fifteen years, and I’m still not as good — nowhere near as good, mind you — as he was then. I still marvel at his work, and his shocking writing style, when I’m grubbing around in the guts of the Emacs-Lisp byte-compiler.

It makes you wonder how many of him there are out there. You know, programmers at that level. He can’t be the only one. What do you suppose they’re all working on? Or do they all eventually make 25th level and opt for divine ascension ?

In any case, I’m sad that I have to write the obit on one of his greater achievements. Sorry, man. Keep up the cat blog.

Forking XEmacs

I have to include a teeny history lesson. Bear with me. It’s short.

XEmacs was a fork of the GNU Emacs codebase, created about 17 years ago by a famous-ish startup called Lucid Inc. , which, alas, went Tango Uniform circa 1994. As far as I know, their two big software legacies still extant are a Lisp environment now sold by LispWorks , and XEmacs.

I’d also count among their legacies an absolutely outstanding collection of software essays called Patterns of Software , by Lucid’s founder, Richard P. Gabriel . I go back and re-read them every year or so. They’re that good.

Back when XEmacs was forked, there were some fireworks. Nothing we haven’t seen many times before or since. Software as usual. But there was a Great Schism. Nowadays it’s more like competing football teams. Tempers have cooled. At least, I think they have.

As for the whole sordid history of the FSF-Emacs/XEmacs schism, you can read about it online. I’m sure it was a difficult decision to make. There are pros and cons to forking a huge open-source project. But I think it was the right decision at the time, just as decomissioning it is the right decision today, seventeen years later.

XEmacs dragged Emacs kicking and screaming into the modern era. Among many other things, XEmacs introduced GUI widgets, inline images, colors in terminal sessions, variable-size fonts, and internationalization. It also brought a host of technical innovations under the hood. And XEmacs has always shipped with a great many more packages than GNU Emacs, making it more of a turnkey solution for new users.

XEmacs was clearly an important force helping to motivate the evolution of GNU Emacs during the mid- to late-1990s. GNU Emacs was always playing catch-up, and the dev team led by RMS (an even more legendary hacker-hero) complained that XEmacs wasn’t really playing on a level field. The observation was correct, since XEmacs was using a Bazaar -style development model, and could move faster as a direct consequence.

A lot of people were switching over to XEmacs by the mid-1990s: the fancy widgets and pretty colors attracted GNU Emacs users like moths to a bug-zapper.

Problem was, it could actually zap you.

The downside of the Bazaar

I personally tried to use XEmacs many times over a period of many years. I was jealous of its features.

However, I never managed to use XEmacs for very long, because it crashed a lot. I tried it on every platform I used between 1996 and 2001, including HP/UX, SunOS, Solaris, Ultrix, Linux, Windows NT and Windows XP. XEmacs would never run for more than about a day under moderate use without crashing.

I’ve argued previously that one of the most important survival traits of a software system is that it should never reboot . Emacs and XEmacs are at the leading edge of living software systems, but XEmacs has never been able to take advantage of this property because even though it can live virtually forever, it’s always tripping and falling down manholes.

Clumsy XEmacs. Clumsy!

I assume its propensity for inopportune heart attacks is a function of several things, including (a) old-school development without unit tests, (b) the need to port it to a gazillion platforms, including many that nobody actually uses, (c) a culture of rapid addition of new features. There are probably other factors as well.

I’m just speculating though. All I know is that it’s always been very, very crashy. It doesn’t actually matter what the reasons are, since there’s no excuse for it.

Interestingly, most XEmacs users I’ve talked to say they

```
}
{
```

How async/await works internally in Swift

Bruno Rocha · SwiftRocks

async/await in Swift was introduced with iOS 15, and I would guess that at this point you probably already know how to use it. But have you ever wondered how async/await works internally? Or maybe why it looks and behaves the way it does, or even why was it even introduced in the first place?

In typical SwiftRocks fashion, we're going deep into the Swift compiler to answer these and other questions about how async/await works internally in Swift . This is not a tutorial on how to use async/await; we're going to take a deep dive into the feature's history and implementation so that we can understand how it works, why it works , what you can achieve with it, and most importantly, what are the gotchas that you must be aware of when working with it.

Disclaimer: I never worked at Apple and have nothing to do with the development of async/await. This is a result of my own research and reverse-engineering, so expect some of the information presented here to not be 100% accurate.

Swift, and the goal of memory safety

Swift's async/await brought to the language a brand new way of working with asynchronous code. But before trying to understand how it works, we must take a step back to understand why Swift introduced it in the first place.

The concept of undefined behavior in programming languages is something that surely haunts anyone who ever needed to work with the so-called "precursor programming languages" like C++ or Obj-C.

Programming languages historically provided you with 100% freedom. There were no real guardrails in place to prevent you from making horrible mistakes; you could do anything you wanted, and the compilers would always assume that you knew what you were doing.

On one hand, this behavior on behalf of the languages made them extremely powerful, but on the other hand, it essentially made any piece of software written with them a minefield. Consider the following C code where we attempt to read an array's index that doesn't exist:

/\/ arr only has two elements void readArray(int arr[]) { int i = arr[20]; printf("%i", i); }

We know that in Swift this will trigger an exception, but we're not talking about Swift yet, we're talking about a precursor language that had complete trust in the developer. What will happen here? Will this crash? Will it work?

The short answer is that we don't know. Sometimes you'll get 0, sometimes you'll get a random number, and sometimes you'll get a crash. It completely depends on the contents of that specific memory address will be at that specific point in time, so in other words, the behavior of that assignment is undefined.

But again, this was intentional. The language assumed that you knew what you were doing and allowed you to proceed with it, even though that turned out to almost always be a huge mistake.

Apple was one of the companies at the time that recognised the need for a safer, modern alternative to these languages. While no amount of compiler features can prevent you from introducing logic errors, they believed programming languages should be able to prevent undefined behavior, and this vision eventually led to the birth of Swift: a language that prioritized memory safety.

One of the main focuses of Swift is making undefined behavior impossible, and today this is achieved via a combination of compiler features (like explicit initialization, type-safety, and optional types) and runtime features (like throwing an exception when an array is accessed at an index that doesn't exist. It's still a crash, but it's not undefined behavior anymore because now we know what's supposed to happen!).

You could argue that this should come with the cost of making Swift an inferior language in terms of power and potential, but one interesting aspect of Swift is that it still allows you to tap into that raw power that came with the precursor languages when necessary. These are usually referred to within the language as "unsafe" operations, and you know when you're dealing with one because they are literally prefixed with the "unsafe" keyword.

let ptr: UnsafeMutablePointer = ...

The problem of concurrency in Swift

But despite being designed for memory safety, Swift was never truly 100% memory safe because concurrency was still a large source of undefined behavior in the language.

The primary reason why this was the case is because Grand Central Dispatch (GCD), Apple's main concurrency solution for iOS apps, was not a feature of the Swift compiler itself, but rather a C library (libdispatch) that was shipped into iOS as part of Foundation. Just as expected of a C library, GCD gave you a lot of freedom in regard to concurrency work, making it challenging for Swift to prevent common concurrency issues like data races, race conditions, deadlocks, priority inversions, and thread explosion.

(If you're not familiar with one or more of the terms above, here's a quick glossary:)

Data Race: Two threads accessing shared data at the same time, leading to unpredictable results

Race Condition: Failing at synchronize the execution of two or more threads, leading to events happening in the wrong order

Deadlock: Two threads waiting on each other, meaning neither is able to proceed

```
}
{
```

A programmer's view of the Universe, part 2: Mario Kart

Steve Yegge

This is the second installment of a little series of discussions. They're not much more than that, just discussions. And I hope I'm inviting discussion rather than quenching it. But I'll be honest — the goal of this series is to pound a stake through the heart of a certain way of thinking about the world that has become quite popular. If my series fails in that regard, I hope it may still provide some entertainment value.

Part one, The fish , was about a twisty line and a fish's examination of it. Today we move to a twisty plane.

There are many kinds of computer programs, and many ways to categorize them. One of the broadest and most interesting program categories is embedded systems . These systems are the centerpiece of today's little chat.

Embedded systems are a bit tricky to define because they come in so many shapes and sizes. Loosely speaking, an embedded system is a little world of its own: an ecosystem with its own rules and its own behavior. So an embedded system need not even be a computer program: a fish tank is also a kind of embedded system.

We call them embedded systems because they exist within the context of a host system . The host system provides the machinery that allows the embedded system to exist, and to do whatever it is that the embedded system likes to do.

For fish tanks, the host system is the tank itself, which you may purchase from a pet store. A tank has walls for holding the water in, filters and pumps for keeping the water clean, lights for keeping the fish and plants alive a little longer, and access holes for putting your hand through to clean the tank. There's not much to it, really. The embedded system is everything inside: the water, the plants, the rocks, the fish, and the little treasure chest with bubbles coming out.

For computer games, another popular kind of embedded system, the host system is the computer that runs the game: a PC, a game console, a phone, anything that can make a game exist for a while so that you may play it.

Programmers have been building embedded systems for many decades: a whole lifetime now. It is a well-studied, well-understood, well-engineered subject. Gamers understand many of the core principles intuitively; programmers, even more so. But in order to apply all that knowledge outside the domain of embedded systems, we will need some new names for the core ideas.

The most important name is the One-Way Wall. I do not have a better name for it. It is the most important concept in embedded systems. In lieu of a good name, I will explain it to you, and then the name will stand for the thing you now understand. It's the best I could come up with.

But first let's dive into an embedded system and see what this wall looks like from the inside.

I will assume you've played Mario Kart , or you've at least watched someone else play it. Mario Kart is the ultimate racing game in terms of sacrificing realism for fun. It bears so little resemblance to reality that it's a wonder it tickles our senses at all. The Karts skid around with impossible coefficients of friction, righting themselves from any wrong position, and generally making a mockery of the last four centuries of advances in physics.

It's really fun.

Mario Kart, like all games, has a boundary around the edge of the playing area. In Mario Kart you bump into it more often than in most other games, which is part of the reason I chose it to be our Eponymous Hero. If you want to win a race, you will need to become quite good at accelerating around corners, which means you will spend a fair amount of time bumping up against an invisible wall.

You know the wall I'm talking about, right? It's invisible, so you can see right past it to the terrain beyond. But the wall is there, and you are not permitted to venture beyond it.

In slower-paced games, when you arrive at the invisible map boundary, you will sometimes be told by the game: "You can't go that way. Turn back!" And since that is your only option, you comply. These invisible boundaries are non-negotiable.

In other games, you may stop on contact with the boundary, or perhaps bounce off. But the boundary is always there.

Imagine Mario and Luigi chatting about the you-can't-go-that-way wall. Their conversation might go something like this:

Mario: "Luigi, my brother!"

Luigi: "Maaaarioooooo!"

Mario: "Yes, Luigi. I am a here. Tell me my brother, why is it that every a time I a spin around the cactus in the third a bend of the Desert a Track, I get a stuck and have to start accelerating from nooooothing?"

Luigi: "Whaaaaa?"

Mario: "Brother, the Desert a Track! It's Number a Three! You know the big a bend, where you have to slow down? I am always a forgetting to slow, and I just a stop. Just a like that!"

Luigi: "I don't know, Brother. That Bowser, he is always a squishing me right before I hit that turn, and I am a flat like a pan a cake for a looooong a time."

Mario: "What about that a time two races ago, where Wario hit you and you a spun around and you a headed for the hill, and you got a stuck and wailed for me?"

```
}
{
```

Software engineering book recommendations

Bruno Rocha · SwiftRocks

The following is a list of software engineering books I've read that I felt had a strong and lasting positive impact on my career. It's not a list of everything I enjoyed (that would be impossible to list down), but rather a special list of resources that taught/helped me so much that I still find myself thinking about them years later. They are my top recommendations for other software engineers.

Some are about iOS development specifically, but most relate to general software engineering. If you strive to be a world-class developer, these books and resources will help you get there.

Write Great Code: Understanding The Machine (Randall Hyde) - Probably my favorite software engineering book. This book teaches you how modern computers work in a high-level way that is easy to understand and not overly technical. It's not going to teach you new fancy APIs, but it will give you the ability to "understand" code; an ability that has proved to be useful almost daily in my career.

Operating Systems: Three Easy Pieces (Remzi Arpaci-Dusseau, Andrea Arpaci-Dusseau) - This is similar to Understanding The Machine, but is more technical and focused on operating systems specifically. Amazing resource to learn about core OS fundamentals such as memory, threads / concurrency, file systems, and CPU virtualization. It's also free.

Networking for Systems Administrators (Michael W Lucas) - Having some basic knowledge of how the modern internet works is really handy regardless of your field of expertise. There are many resources out there that teach will you networking, but they also assume that you work professionally with it, so they spend too much time explaining implementation details that I didn't care about. I have then always been searching for a book that could teach me how networking works at a basic level , without wasting time on those deeper details, and this book was able to fit that criteria perfectly. Word of caution: If you're anything like me, you're going to go down the Homelab rabbit hole after reading this book!

*OS Internals Trilogy (Jonathan Levin) - This trilogy about how Apple platforms work internally is an absolute gold mine for us who develop for said platforms, but it's not for everyone. They will teach you everything you could possibly want to know about these platforms, but they're massive and go into extreme detail on things that you're never going to interact with, making this series a very painful read for beginners and those looking for quick / practical knowledge. But if you're reasonably experienced and is interested in things like linkers and kernels, then these books are a godsend. The only problem with these books is that Amazon only delivers them if you live in the US, so getting a copy can be challenging.

The Art Of Mac Malware, Volume 1 (Patrick Wardle) - A really interesting overview of the different tricks that malware authors have used to infect macOS machines in recent years, as well as a great introduction to disassemblers and advanced debugging techniques. Amazing if you'd like to get better at reverse engineering / debugging and expand your knowledge of how macOS works under the hood. There is also a second volume, but I haven't read that yet.

Game Engine Black Book: DOOM (Fabien Sanglard) - Fantastic deep-dive on not only the source code, but also the hardware, tooling, and team dynamics that allowed the masterpiece known as DOOM to be conceived. iOS developers will find this book extra interesting because DOOM was developed on NeXT workstations; many of the tools were written in Objective-C, so the book has a lot of interesting information about the language and frameworks like Foundation that is still relevant to this day.

Cracking the Coding Interview (Gayle Laakmann McDowell) - Even if you don't care about the interviewing bits (I'm not even sure if companies like Apple still run LeetCode puzzles), this book is still a legendary resource for getting started with computer science theory. If you're not sure why you'd want to do that, check out my article about it.

The Staff Engineer's Path (Tanya Reilly) - Perfect book for senior engineers looking to get to the next level. This book demystifies the concept of a Staff Engineer and provides a gigantic amount of great advice on how to get there and how to do a good job once you're there.

Effective Objective-C 2.0 (Matt Galloway) - Even though we don't directly write apps in Obj-C anymore, we still have to do so indirectly when using older frameworks like UIKit. This book does a great job of explaining how Obj-C works, which is something you'll wish you knew when dealing with issues that either originate or pass through Obj-C.

Clean Code (Robert C. Martin) - This book gets a lot of flack in the community because some people treat it as some sort of bible that must be followed religiously, but it is nonetheless a great resource for learning how to write code that is easy to read and maintain by humans. Just don't be like those folks.

Hacking: The Art of Exploitation (Jon Erickson) - I didn't actually like the first half of this book as the author spends far too long explaining C / Assembly and C exploits that are probably largely irrelevant nowadays, but the reason I'm adding it to the list is because of the second half which does a pretty good job at explaining how networks work , including some network-based exploitation techniques which I believe are still relevant to this day. This led me to a rabbit hole of wanting to know more about how to reverse engineer online-based apps and games, and thus I'm very grateful for this book even though I wasn't a fan of the first half. I feel then that this book in addition to the previous networking book recommendation gave a pretty solid foundation on networking.

```
}
{
```

The 2025 Go Cryptography State of the Union

Filippo Valsorda

This past August, I delivered my traditional Go Cryptography State of the Union talk at GopherCon US 2025 in New York.

It goes into everything that happened at the intersection of Go and cryptography over the last year.

You can watch the video (with manually edited subtitles, for my fellow subtitles enjoyers) or read the transcript below (for my fellow videos not-enjoyers).

style="text-align: center;">

The annotated transcript below was made with Simon Willison's tool . All pictures were taken around Rome, the Italian contryside, and the skies of the Northeastern United States.

id="annotated-transcript">Annotated transcript

#

Welcome to my annual performance review.

We are going to talk about all of the stuff that we did in the Go cryptography world during the past year.

#

When I say "we," it doesn't mean just me, it means me, Roland Shoemaker, Daniel McCarney, Nicola Murino, Damien Neil, and many, many others, both from the Go team and from the Go community that contribute to the cryptography libraries all the time.

I used to do this work at Google, and I now do it as an independent as part of and leading Geomys , but we'll talk about that later.

#

When we talk about the Go cryptography standard libraries, we talk about all of those packages that you use to build secure applications.

That's what we make them for. We do it to provide you with encryption and hashes and protocols like TLS and SSH, to help you build secure applications .

#

The main headlines of the past year:

We shipped post quantum key exchanges, which is something that you will not have to think about and will just be solved for you.

We have solved FIPS 140, which some of you will not care about at all and some of you will be very happy about.

And the thing I'm most proud of: we did all of this while keeping an excellent security track record, year after year.

#

This is an update to something you've seen last year.

The Go Security Track Record

It's the list of vulnerabilities in the Go cryptography packages.

We don't assign a severity—because it's really hard, instead they're graded on the "Filippo's unhappiness score."

It goes shrug, oof, and ouch.

Time goes from bottom to top, and you can see how as time goes by things have been getting better. People report more things, but they're generally more often shrugs than oofs and there haven't been ouches.

#

More specifically, we haven't had any oof since 2023.

We didn't have any Go-specific oof since 2021.

When I say Go-specific, I mean: well, sometimes the protocol is broken, and as much as we want to also be ahead of that by limiting complexity, you know, sometimes there's nothing you can do about that.

And we haven't had ouches since 2019 . I'm very happy about that.

#

But if this sounds a little informal, I'm also happy to report that we had the first security audit by a professional firm.

Trail of Bits looked at all of the nuts and bolts of the Go cryptography standard library: primitives, ciphers, hashes, assembly implementations. They didn't look at the protocols, which is a lot more code on top of that, but they did look at all of the foundational stuff.

And I'm happy to say that they found nothing .

Two of a kind t-shirts, for me and Roland Shoemaker.

#

It is easy though to maintain a good security track record if you never add anything, so let's talk about the code we did add instead.

First of all, post-quantum key exchanges.

We talked about post-quantum last year, but as a very quick refresher:

Post-quantum cryptography is about the future. We are

```
}
{
```

A Retrospective Survey of 2024/2025 Open Source Supply Chain Compromises

Filippo Valsorda

Lack of memory safety is such a predominant cause of security issues that we have a responsibility as professional software engineering to robustly mitigate it in security-sensitive use cases—by using memory safe languages.

Similarly, I have the growing impression that software supply chain compromises have a few predominant causes which we might have a responsibility as a professional open source maintainers to robustly mitigate.

To test this impression and figure out any such mitigations, I collected all 2024/2025 open source supply chain compromises I could find, and categorized their root cause. (If you find more, do email me!)

Since I am interested in mitigations we can apply as maintainers of depended-upon projects to avoid compromises, I am ignoring: intentionally malicious packages (e.g. typosquatting), issues in package managers (e.g. internal name shadowing), open source infrastructure abuse (e.g. using package registries for post-compromise exfiltration), and isolated app compromises (i.e. not software that is depended upon).

Also, I am specifically interested in how an attacker got their first unauthorized access, not in what they did with it. Annoyingly, there is usually a lot more written about the latter than the former.

id="20242025-open-source-supply-chain-compromises">2024/2025 Open Source Supply Chain Compromises

In no particular order, but kind of grouped.

XZ Utils

Long term pressure campaign on the maintainer to hand over access.

Root cause : control handoff.

Contributing factor: non-reproducible release artifacts.

Nx S1ingularity

Shell injection in GitHub Action with pull_request_target trigger and unnecessary read/write permissions 1 , used to extract a npm token.

Root cause : pull_request_target.

Contributing factors: read/write CI permissions, long-lived credential exfiltration, post-install scripts.

Shai-Hulud

Worm behavior by using compromised npm tokens to publish packages with malicious post-install scripts, and compromised GitHub tokens to publish malicious GitHub Actions workflows.

Root cause : long-lived credential exfiltration.

Contributing factor: post-install scripts.

npm debug/chalk/color

Maintainer phished with an “Update 2FA Now” email. Had TOTP 2FA enabled.

Root cause : phishing.

polyfill.io

Attacker purchased CDN domain name and GitHub organization.

Root cause : control handoff.

MavenGate

Expired domains and changed GitHub usernames resurrected to take control of connected packages.

Root causes : domain resurrection, username resurrection.

reviewdog and tj-actions/changed-files

Contributors deliberately granted automatic write access for GitHub Action repository 2 . Malicious tag re-published to compromise GitHub PAT of more popular GitHub Action 3 .

Root cause : control handoff.

Contributing factors: read/write CI permissions, long-lived credential exfiltration, mutable GitHub Actions tags.

Shell injection in GitHub Action with pull_request_target trigger (which required read/write permissions), pivoted to publishing pipeline via GitHub Actions cache poisoning. Compromised again later using an exfiltrated PyPI token.

Root cause : pull_request_target.

Contributing factors: GitHub Actions cache poisoning, long-lived credential exfiltration.

GitHub Action with pull_request_target trigger restricted to trusted users but bypassed via Dependabot impersonation 4 , previously patched but still available on old branch. GitHub PAT exfiltrated and used.

Root causes : pull_request_target, Dependabot impersonation.

Contributing factors: per-branch CI configuration, long-lived credential exfiltration.

with: ref: refs/pull/${{ github.event.issue.number }}/head

Pwn request 5 against issue_comment workflow 6 in other project, leading to a GitHub classic token of a maintainer

```
}
{
```

Things that did (and didn't) contribute to Burnout Buddy's success

Bruno Rocha · SwiftRocks

Back in 2022 I launched Burnout Buddy , and today the app has succeeded far beyond my expectations. Netting between $600 and $1000 each month as of writing, BB has been growing 100% organically with little to no effort on my part.

In this post, I'd like to lay out exactly what I've done that I believe contributed (and didn't contribute) to this growth, serving as documentation and inspiration for the indie dev community out there.

Things that helped

Understanding ASO

I cannot understate the value of having a good grasp of App Store Optimization (ASO) . The case is simple: It doesn't matter how good your app is, if you don't get eyes on it, it will never succeed.

ASO refers to being strategic about how you assemble your app's store listing (keywords, name, subtitle, description, screenshots, etc) so that it ranks well when people search for keywords related to your app. In many cases what you actually want to do is avoid popular keywords in the beginning, focusing on less popular ones where you have more of a fighting chance until you get "popular" enough that you can try challenging the real ones. How and when you ask for reviews also plays a big role here as reviews also affect your app's rank.

I strongly recommend Appfigures for learning and applying ASO for your apps. The owner, Ariel, has posted many videos explaining different strategies you can take, and that's how I got to know about it.

In my case, ASO was only time-intensive in the first few weeks following the app's launch. After it picked up some steam and became no.1 in a couple of important keywords, I was able to leave it alone and enjoy full organic growth ever since.

I'm my app's primary user

Most indie apps fail because they are trying to solve problems that don't exist. The devs come up with the solution first , and then try to find users who have a problem that match their solution. This rarely works.

The easiest way to avoid this is to ignore other people and just focus on your own set of problems. If you can manage to build something that would make your own life better, certainly you'll find other people who will also appreciate it.

In my case, I built Burnout Buddy because iOS's default Screen Time feature was too simple for me. I wanted to make more complex scenarios such as schedule or location based conditions, but iOS only allows you to setup simple time limits. You also can't do "strict" conditions where there's no way to disable the block once it goes into effect. I searched for other alternatives, but none of them were good enough for me. So I built my own!

Once my problem was solved, I figured out that most likely there were others out there who could also make use of it. I made the app public with zero expectations, and sure enough, there were tons of other people with the same problem I had.

Being my app's primary user also means that I'm perfectly positioned to know which features the app should and shouldn't have. I don't need things like user interviews, because again, I built this for myself. All I have to do is ask myself what I'd like the app to do, and the result is sure to also be a hit with others with the same problem the app aims to solve.

I attribute Pieter Level's Make book for helping me understand this concept. It's also a great resource for learning more about indie development and how to create successful products in general!

No backend, everything happens client-side

Another decision that I've made that massively simplified things for me is that everything happens on the client. There are no accounts or backend, and I gather zero data from the users.

This means I have no backend to manage, and most importantly, no monthly server costs. As long as Apple doesn't push iOS updates that break the APIs I use (unfortunately happens a lot) , I can trust that everything is working as it should and focus my attention on other things. People seem to really appreciate this too, since many apps nowadays have accounts for no reason other than wanting to hoard data which is really shady.

The app just works

After the first couple of releases, I spent a good amount of time building a good suite of tests and architecting the app so that it would be easy to expand and make it even more testable. This means I very rarely have to worry about whether or not I'll push something that will fundamentally break the app. Having no backend-related code also greatly helped here.

This doesn't mean that the app is bug-free (there are a bunch of SwiftUI issues I can't seem to solve, and Apple somehow manages to break their APIs on every iOS release as mentioned above), but when it comes to the core experience of the app, I can trust that everything works as it should. This saved a lot of testing / debugging time on my end and also made sure I almost never had to deal with support e-mails regarding broken features and such.

I don't extort my users

}

standard-article(title: [What are mobile release engineering teams and when do you need one? (Runway)], author: [Bruno Rocha], source-name: [SwiftRocks], images: (), paragraphs: ([The early (and also not-so-early) days of building a tech startup means hiring and working with people who are capable of wearing a lot of hats. There are a ton of things to do and nowhere near an equal amount of resources to do them. Even if you join as a specialist, chances are you'll find yourself getting deep into other areas.], [This is true across the organization, including in mobile engineering. Early engineers don't just write the code that builds the foundation for the app's future success (while building up the tech debt that future engineers will pull their hair out over) but also the initial infrastructure both for how teams get things done and how they ship code out the door.], [This doesn't last forever though. As teams grow they make the transition out of generalist, jack-of-all-trades roles and begin focusing on specialization. But one part of the org where there is often a lag in making this transition — and sometimes a very long one — is on the mobile team. Very few mobile engineers will have time to support both their team's app features and the series of Ruby scripts that support their release process, yet the expectation that they do both often lingers far beyond when other teams in the organization have fully specialized.], [In this article, we'll look at why this happens and how big tech companies follow-up on this problem.],), insert-map: (:), word-count: 237, edited-for-length: false, debug-mode: false,)

brief-group(([

John Gruber, Daring Fireball: Here's a post from 2015, linking to Rene Ritchie, then still at iMore, explaining how iMore found itself serving ever worse (and more reader hostile) ads. Not much has changed regarding the state of web advertising in a decade, and iMore — once a truly great site — is defunct .

⋆

], [

Steef-Jan Wiggers, InfoQ: Discord engineering detailed how they added distributed tracing to Elixir's actor model. Their custom Transport library wraps messages with trace context and uses dynamic sampling to handle million-user fanouts. CPU optimizations included skipping unsampled traces and filtering context before deserialization, recovering 10+ percentage points of overhead.

By Steef-Jan Wiggers

], [

Daring Fireball Department of Commerce, Daring Fireball: Video isn't just something to watch; it's a boatload of context and data. Mux makes it easy to ship and scale video into anything from websites to platforms to AI workflows. Unlock what's inside: transcripts, clips, and storyboards to build summarization, translation, content moderation, tagging, and more.

Mux stewards Video.js, the web's most popular open source video player. Video.js v10 is a complete architectural rebuild, with the beta now available at videojs.org .

Mux is video infrastructure trusted by Patreon, Substack, and Synthesia. Get started free, no credit card required. Use code FIREBALL for an extra $50 credit.

⋆

], [

Bruno Couriol, InfoQ: The new, experimental Web Install API is now in Origin Trial in Microsoft Edge and Chrome. The API allows developers to programmatically trigger a PWA installation prompt from in-app user interactions. The API aims to simplify software discovery and distribution, particularly for users who are unaware of the install icon in the browser's address bar or do not typically use app stores.

By Bruno Couriol

],))

The Report

Vol. 1, No. 034 · 2026-03-30

Curated and composed automatically.